SMARTLOCK

Assignment 2:

FIELD	SUBMISSION DETAIL
Course	CS3002 - Information Security
Student	Zohaib Umer
Roll / Section	23i-0528 / F

Build commands

make -C smartlock

make -C v2_skeleton

g++ -std=c++17 -O2 -Wall -Wextra -I smartlock [A2-A5.cpp] -o tool -lcrypto

Scope, ethics, and laboratory method

All interception and modification exercises were conducted for an assigned, deliberately insecure SmartLock instance in an isolated VirtualBox Internal Network. VM1 hosted the lock and VM2 hosted the hub, relay, and packet-capture tools. The attacker had network visibility and the ability to relay or alter protocol traffic but no legitimate access to the lock's private key or configuration secrets for the capture-based recovery demonstrations.

HOST / TOOL	CONFIGURATION
VM1	Kali Linux - SmartLock server at 10.10.10.11:9283
VM2	Kali Linux - hub and controlled relay at 10.10.10.12; relay 127.0.0.1:8000
Transport	TCP, newline-delimited TAB-separated key:value fields
Capture	tcpdump and Wireshark Follow TCP Stream, including lo and eth0 for relay tests
Attack sources	C++17 Part A tools and a documented mitmproxy addon for A1

Normal captured session

The baseline v1 session offered AES256-GCM, AES128-GCM and NULL; the lock selected AES256-GCM. The capture contains HELLO, SUITE, CERT, PAIR, DH, DHCLIENT and encrypted MSG records. The normal hub completed STATUS, UNLOCK, LOCK and QUIT.

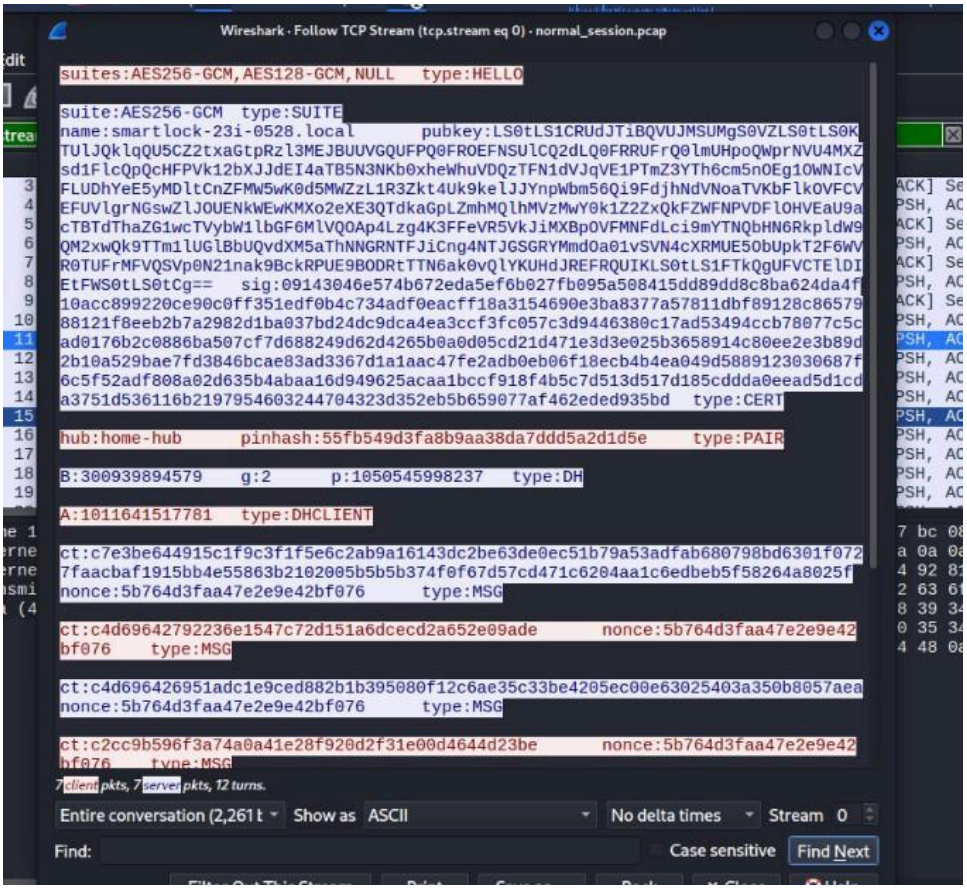


Figure 1. Baseline VM2 Wireshark capture

Part A - A1: Negotiation downgrade

Weakness and attack method

The v1 hub includes NULL among its offered suites, and negotiation is not authenticated. A mitmproxy relay on VM2 changed the hub's original HELLO offer to suites:NULL on the server-facing connection. The server accepted it and returned suite:NULL.

The test used two capture points: loopback port 8000 for the original hub offer and eth0 port 9283 for traffic forwarded to the lock. This separates the attacker's modification from a legitimate client request. The submitted A1 tool sequence and addon describe the exact relay command and text replacement.

EVIDENCE	OBSERVED
Original HELLO	suites:AES256-GCM,AES128-GCM,NULL
Altered HELLO	suites:NULL
Selected response	suite:NULL
Impact	Confidentiality failure: the NULL record ciphertext field carries unencrypted plaintext bytes encoded as hex

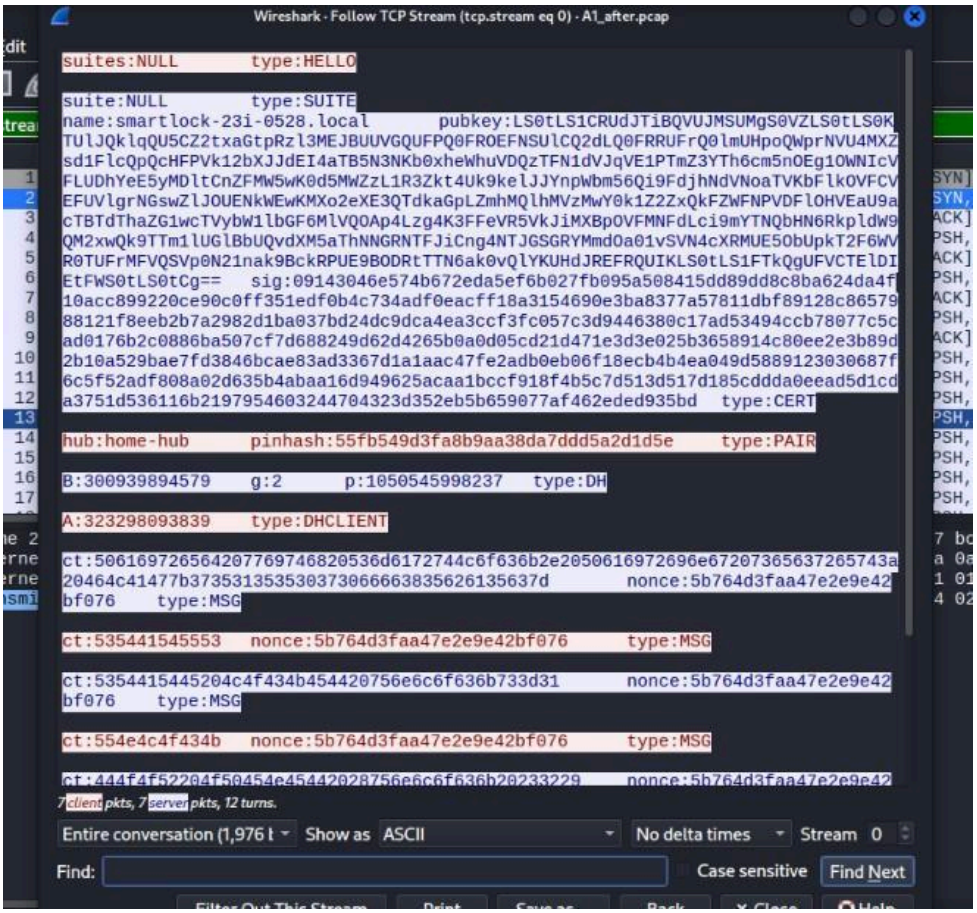


Figure 2. Actual v1 server-facing TCP stream shows NULL-only offer and accepted suite.

Security property: negotiation integrity and confidentiality. The normal client was not intentionally configured for NULL; a network relay downgraded it.

Part A - A2: Impersonation and two-key MITM

The v1 hub displays the device certificate without validating its identity or signature. The C++17 relay provided an attacker-chosen certificate name to the hub, and ran independent DH negotiations with the hub and genuine lock. This created two record keys, allowing the relay to decrypt a protected command and re-encrypt it for the other side.

ACTUAL RELAY LOG FIELD	OBSERVED
Hub-facing key SHA256 prefix	cf768ff16543a8d1
Lock-facing key SHA256 prefix	c862dcf06ec923b9
Key comparison	Keys differ: YES
Readable commands	STATUS, UNLOCK, LOCK, QUIT
Server response	DOOR OPENED (unlock #4); replies read by relay

The packet archives A2_hub_side.pcap and A2_lock_side.pcap contain the two distinct TCP connections and DH public values. A2_relay_output.txt provides the independently observed record translation; A2_hub_output.txt confirms the hub trusted attacker-test-device.local and completed the session.

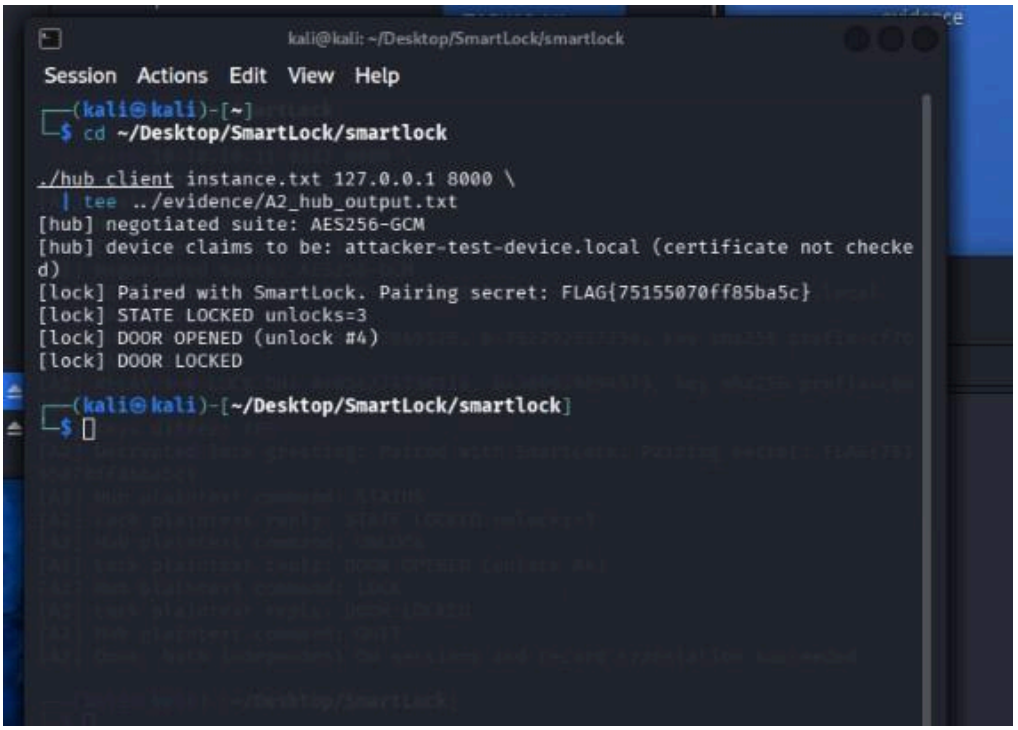


Figure 3. Genuine VM2 evidence: hub accepted the substituted identity and completed commands.

Security property: entity authentication and message confidentiality. This attack requires lack of certificate validation and independent key establishment; changing the displayed name alone would not establish full MITM impact.

Part A - A3: Diffie-Hellman key recovery

The v1 DH group is small enough to solve using baby-step giant-step (BSGS). The C++17 recovery tool parsed the captured DH public parameters and encrypted greeting from A3_tcp.tsv, recovered the device private exponent, verified the modular relation, derived the v1 record key, and authenticated/decrypted the first server greeting.

CAPTURE / COMPUTATION	VALUE
p	1050545998237
g	2
Captured B	300939894579
Captured A	1011641517781
Recovered b	35932352885
Mathematical verification	pow(2, 35932352885, 1050545998237) = 300939894579
Recovered greeting flag	FLAG{75155070ff85ba5c}

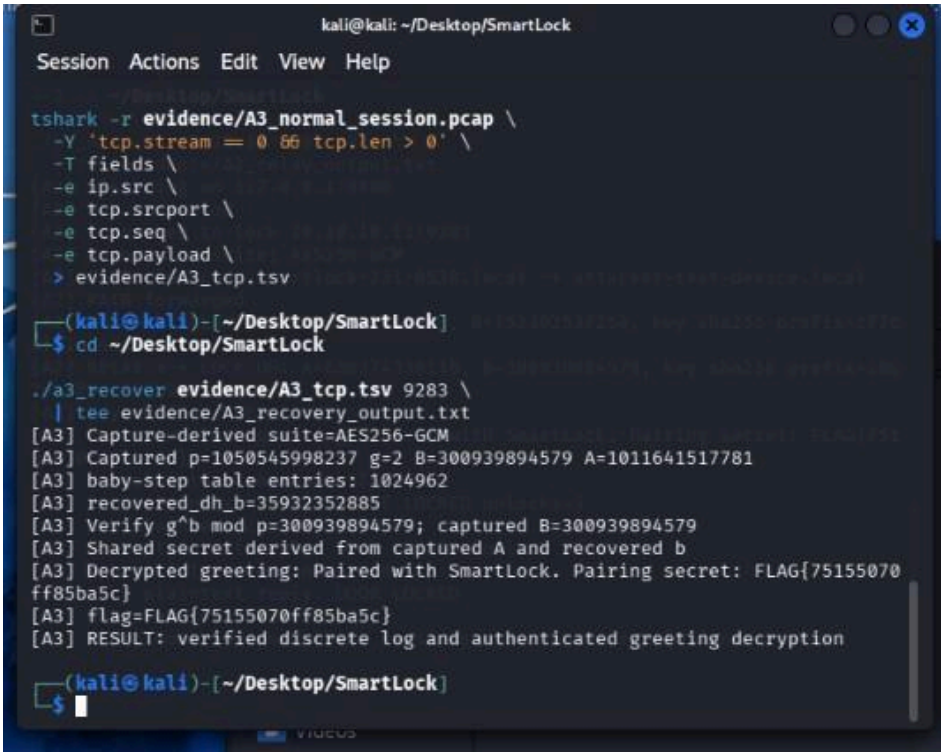


Figure 4. VM2 terminal showing capture-derived discrete log, verification, and decrypted flag.

Security properties: confidentiality and resistance to key recovery. This attack demonstrates why deriving a symmetric key from a secret based on a weak DH group is insufficient.

Part A - A4: GCM nonce reuse

SmartLock v1 encrypts multiple server-to-hub records under one session key and a static 96-bit nonce. When GCM reuses a key/nonce pair, the CTR encryption keystream repeats, and the XOR of ciphertext bodies (excluding 16-byte tags) equals the XOR of the corresponding plaintexts over the overlapping bytes.

C1 xor C2 = P1 xor P2 (same key and nonce; authentication tags excluded)

CAPTURE-BASED RESULT	VALUE
Reused 12-byte nonce	5b764d3faa47e2e9e42bf076
Greeting / STATUS ciphertext sizes	61 bytes / 22 bytes (excluding GCM tags)
Plaintext XOR (hex)	0335282620446c382a3f2d6473180f1e1b2f0410561a
Recovered STATUS prefix	STATE LOCKED unlocks=4

The C++17 analyzer reads the recorded MSG fields in A4_tcp.tsv, removes the trailing 16-byte authentication tags, checks nonce equality, calculates the XOR, and uses a known greeting prefix to infer STATUS plaintext bytes. Evidence is preserved in A4_analysis.txt and A4_normal_session.pcap.

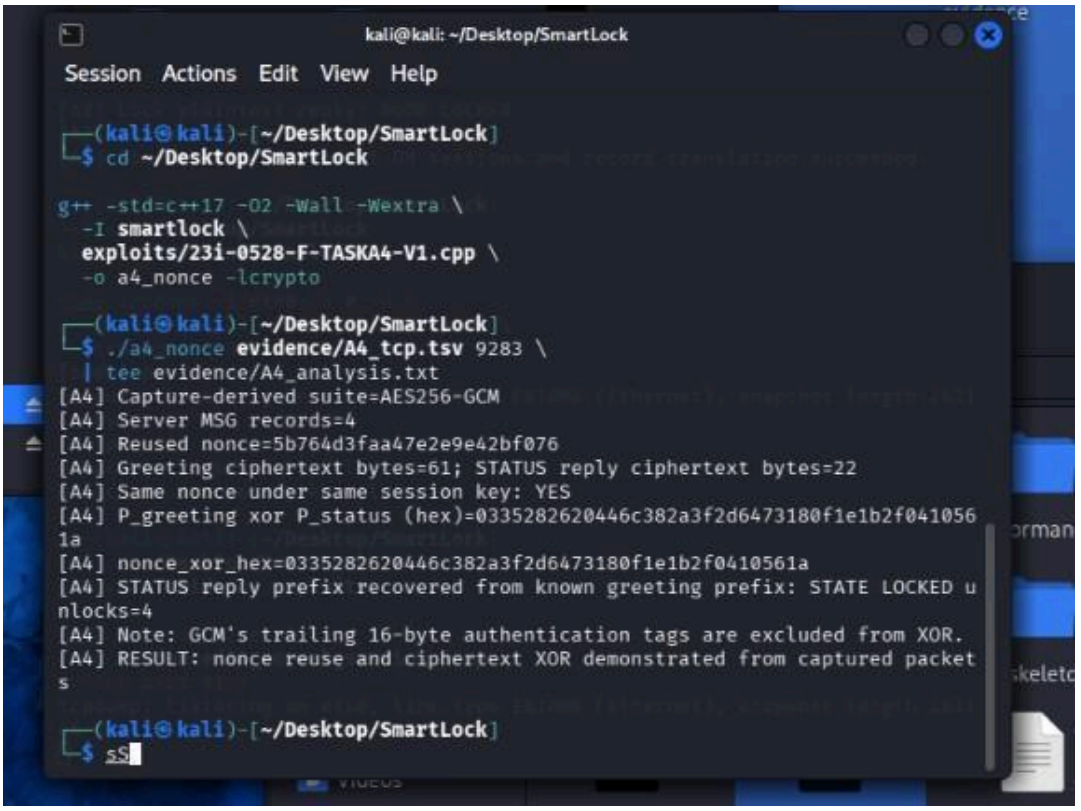


Figure 5. Actual VM2 A4 output reports nonce reuse, recovered XOR and STATUS prefix.

Security property: confidentiality and GCM nonce safety. The attack does not imply the authentication tag itself may be XORed as if it were ciphertext.

Part A - A5: Replay of an encrypted UNLOCK

The stock v1 client emits a deterministic encrypted UNLOCK record using a server key and static nonce that persist across sessions. The C++ replay client captured the initial handshake and encrypted UNLOCK, then opened two new TCP connections and sent the recorded command without modifying its ciphertext.

COUNTER EVIDENCE	OBSERVED
Before baseline session	STATUS: unlocks=5
After legitimate baseline UNLOCK	unlocks=6
After two fresh-session replays	next STATUS: unlocks=8
Difference	+3 total: one legitimate unlock and two replayed unlocks
Server state	STATE UNLOCKED at the next STATUS before another normal UNLOCK

A5_before_hub.txt, A5_replay_output.txt and A5_after_hub.txt contain the independently recorded counter chain. A5_baseline.pcap and A5_replay.pcap capture the original and replay TCP sessions. Retransmitting original TCP packet sequence numbers alone is not equivalent to this fresh-connection replay.

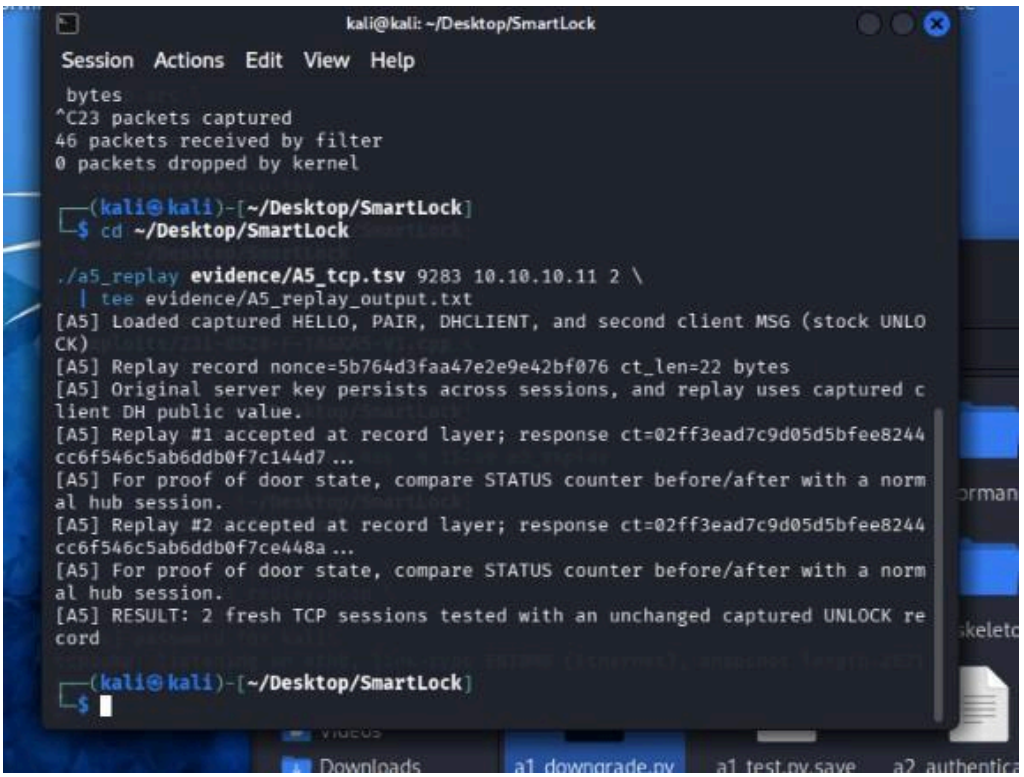


Figure 6. Actual replay verification screenshot; raw counter logs are in evidence/.

Security property: record freshness and command integrity. No anti-replay sequence number or session-bound key invalidated the recorded ciphertext in v1.

Part A.1 - Recovered instance values and verification

The values below correspond to roll number 23i-0528. The flag and DH exponent are supported by the captured A3 decryption log; the nonce XOR by the A4 capture analysis; and the PIN by an offline dictionary+mask check of the captured PAIR digest. The raw instance file is not included in the submission.

KEY IN answers.txt	CAPTURE-DERIVED VALUE
roll	23i-0528
flag	FLAG{75155070ff85ba5c}
recovered_dh_b	35932352885
cracked_pin	turbo681
nonce_xor_hex	0335282620446c382a3f2d6473180f1e1b2f0410561a

PIN recovery evidence

The captured PAIR line exposed pinhash:55fb549d3fa8b9aa38da7ddd5a2d1d5e. The supplied dictionary contained 86 words, each combined with a three-digit suffix (000-999). The match turbo681 was verified by independently computing MD5(turbo681), which yielded the captured digest. The audit file evidence/A1_PIN_capture_recovery.txt records the matching procedure. The MD5 digest remains vulnerable in the supplied v2 scaffold; the Part B design explicitly addresses it as a separate hardening requirement.

Verification and reproducibility

The exponent satisfies the recorded relation: 2^35932352885 modulo 1050545998237 equals captured B = 300939894579. The flag was recovered by decrypting a captured greeting rather than taken from a device file. The nonce XOR is 22 bytes, equal to the relevant STATUS reply plaintext length and therefore within the assignment maximum.

Part B - v2 design: threat model (4 marks)

System assets and assumed adversary

Assets are the authority to unlock the door, the real lock identity and signing key, the pairing credential, session keys, and the confidentiality, authenticity and freshness of each command. The assumed network adversary can capture packets, relay or replace messages, initiate TCP connections, replay old records, and compute offline attacks. The adversary cannot read genuine endpoint memory or private signing-key files, replace trusted software, physically tamper with the lock, or break correctly deployed asymmetric or symmetric primitives. Denial of service remains outside the protocol's confidentiality/authentication guarantees.

GOAL	PART A FAILURE	REQUIRED V2 GUARANTEE
G1 Negotiation integrity	A1 NULL downgrade	No NULL suite; signed and authenticated suite transcript
G2 Device authentication	A2 certificate spoofing	Pinned public key and verified certificate + signed DH transcript
G3 Session confidentiality	A3 small DH discrete log	Fresh 2048-bit DH shared secret and KDF-derived keys
G4 AEAD nonce safety	A4 static nonce	Fresh per-sequence nonce under each directional key
G5 Replay resistance	A5 old UNLOCK accepted	Expected sequence binding and new key per session
G6 PIN protection	Offline MD5 PIN cracking	Design-only PAKE + hardened stored verifier

Part B - Per-stage defensive mechanisms (10 marks)

Stage 1: negotiation versus A1

The hub advertises only AES256-GCM and AES128-GCM; choose_suite() selects the strongest common option and returns empty when none exists. The client verifies the received suite. The server additionally signs the offer|selected-suite|DH-public-value transcript, preventing a relay from silently changing negotiation when authentication completes.

Stage 2: authentication versus A2 and PIN exposure

The lock provides its device name and RSA public key with a signature. The hub compares the key to the pinned key and verifies both the certificate and signed DH transcript. A rogue replacement key or modified certificate name cannot authenticate. Design-only PIN improvement: replace the replayable PAIR digest with a vetted password-authenticated key exchange (e.g. SPAKE2+), bind its confirmation to the session, protect a salted, memory-hard verifier at rest, and rate-limit online pairing. Merely HMACing a low-entropy PIN with a visible challenge still permits offline guessing, so it is not proposed as the final solution.

Stage 3: keying versus A3

Use the provided RFC 3526 2048-bit MODP group with g=2 and fresh 256-bit randomly generated private exponents per connection. Confirm that the presented device public DH value is signed. Both peers calculate the same DH secret, then securelink2.cpp derives independent c2s and s2c AES keys using HKDF-SHA256 and context labels. Erase ephemeral private material after use in a hardened implementation.

Stage 4: record protection versus A4

AES-GCM is used with a nonce derived from a monotonically increasing 64-bit sequence number, and a separate key per direction. seal() includes an eight-byte sequence header, authenticates it as AEAD associated data, appends the 16-byte GCM tag, and open_record() rejects the wrong sequence or invalid tag before releasing plaintext. Distinct sequence numbers guarantee distinct nonces under the same key.

Stage 5: teardown versus A5

Each STATUS/UNLOCK/LOCK/QUIT command is inside a sequence-bound authenticated record. The next expected sequence must match; a replay from the same session is out of order, while an old session record cannot authenticate under a newly derived ephemeral-session key. QUIT is therefore protected and ends the connection; no later command on that session is acted on.

Part B - Actual v2 handshake diagram (4 marks)

Figure 7 follows the provided v2_skeleton code rather than inventing new on-wire message types. Certificate pinning and the signed DH transcript are implemented; stronger PAKE-based PIN protection is labeled as design-only.

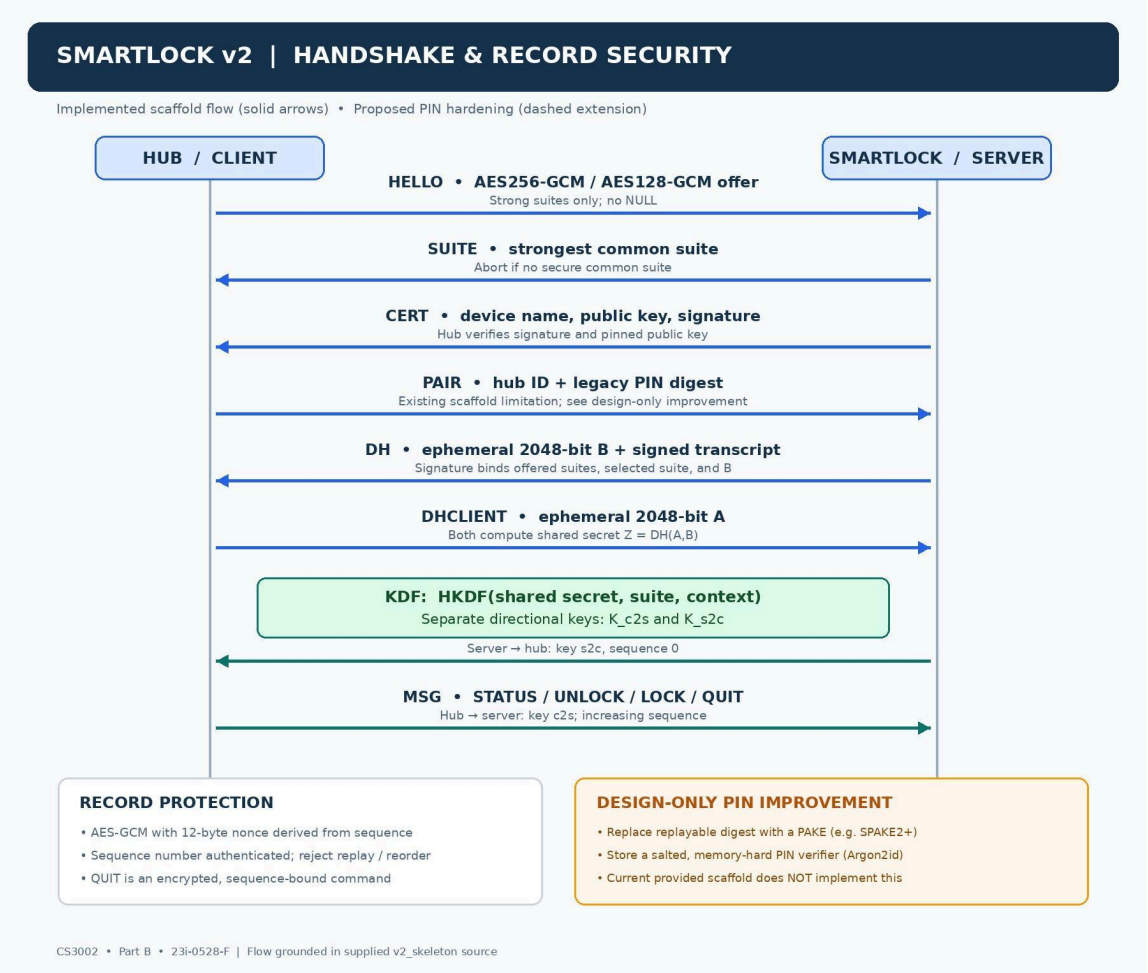


Figure 7. Negotiation, certificate verification, signed ephemeral DH, HKDF, sequence-bound AES-GCM, and protected QUIT.

Diagram source is included as evidence/SmartLock_v2_Handshake_PartB.png. The same method is referenced by Part C implementation and tests.

Part C.1 - Security core implementation (18 marks)

The supplied v2 client and server retain protocol plumbing, certificate pinning, signed ephemeral DH, and the command loop. The submitted securelink2.cpp implements the six required security-core behaviors while preserving securelink2.hpp signatures.

CORE FUNCTION / CHECK	IMPLEMENTED BEHAVIOR
strong_suites / choose_suite	Only AES256-GCM / AES128-GCM; reject NULL-only offer
derive_key	OpenSSL EVP_KDF HKDF-SHA256 over shared DH secret, suite and c2s/s2c context
make_nonce	12 bytes: four fixed bytes followed by big-endian 64-bit sequence
seal / open_record	AES-GCM encrypt/decrypt, 16-byte tag, sequence authenticated as AAD
Replay / order	Compare received 8-byte sequence with expected and throw on mismatch
Source scan	Contains HKDF/EVP_KDF; excludes forbidden md5 and TODO markers

Self-test results

The uploaded VM self-test log contains 11 PASS entries covering suite policy, key derivation, AES128/256 round-trips, nonce uniqueness, replay, ordering, tampered ciphertext, tampered tag, altered sequence and truncated records. The same uploaded module was also rebuilt with g++ and OpenSSL in the report environment.

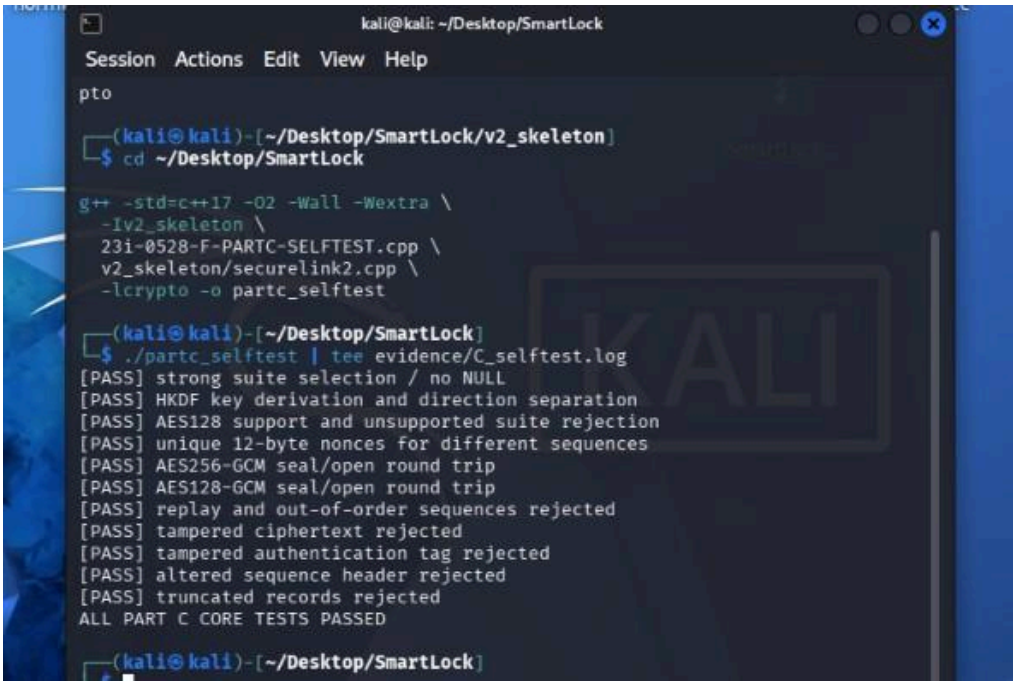


Figure 8. Evidence screenshot for the Part C core self-test; machine-readable

Build command: make -C v2_skeleton. Submitted source file retains exact name securelink2.cpp

Part C.2 - Re-run A1-A5 against v2 (5 marks)

A normal v2 client/server session completed under AES256-GCM with device authenticated. The following table summarizes actual uploaded tests; each entry has a corresponding packet capture or console output.

V1 ATTACK	V2 TEST EVIDENCE	RESULT
A1 Downgrade	C_A1_downgrade_fixed.pcap and C_A1_fixed_output.txt	NULL-only HELLO delivered; hub abort: no suite
A2 MITM	C_A2_hub.pcap; C_A2_hub_output.txt	Substituted certificate rejected: bad certificate signature
A3 Keying	C_A3_group_check.txt; C_A3_shared_check.txt; v2 PCAP	2048-bit DH, fresh 256-bit exponent; practical v1 BSGS recovery does not scale
A4 Nonce reuse	C_A4_nonce_test.txt; C_A4_v2_session.pcap	10,000 distinct 12-byte nonces for tested sequence range; authenticated records
A5 Replay	C_A5_replay.pcap; C_A5_replay_result.txt; before/after logs	

Replay or out-of-order record; no extra unlock

Attack rejection evidence

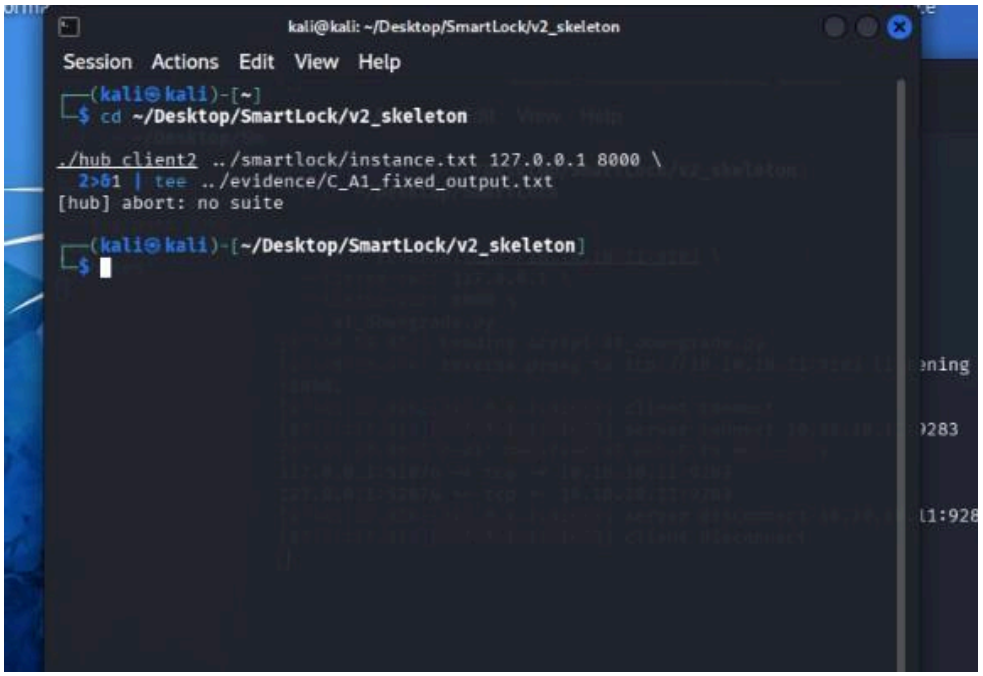


Figure 9. C-A1 tested the v2 downgrade and produced an abort rather than NULL negotiation.

Part C.2 - Correctness and AI-assisted code audit

Normal session, tampering, and replay rejection (2 marks)

The normal v2 hub output authenticates the device and completes a STATUS/UNLOCK/LOCK session. The uploaded self-test rejects modified ciphertext, modified authentication tags, sequence header changes and truncated records. In the network replay test, the hub terminated with Replay or out-of-order record.

A5 COUNTER CHECK	OBSERVED
C_A5_before.txt first STATUS	unlocks=2
Legitimate baseline UNLOCK	unlocks=3
Legitimate UNLOCK during attacked session	unlocks=4
C_A5_after.txt first STATUS	unlocks=4; no additional replay unlock

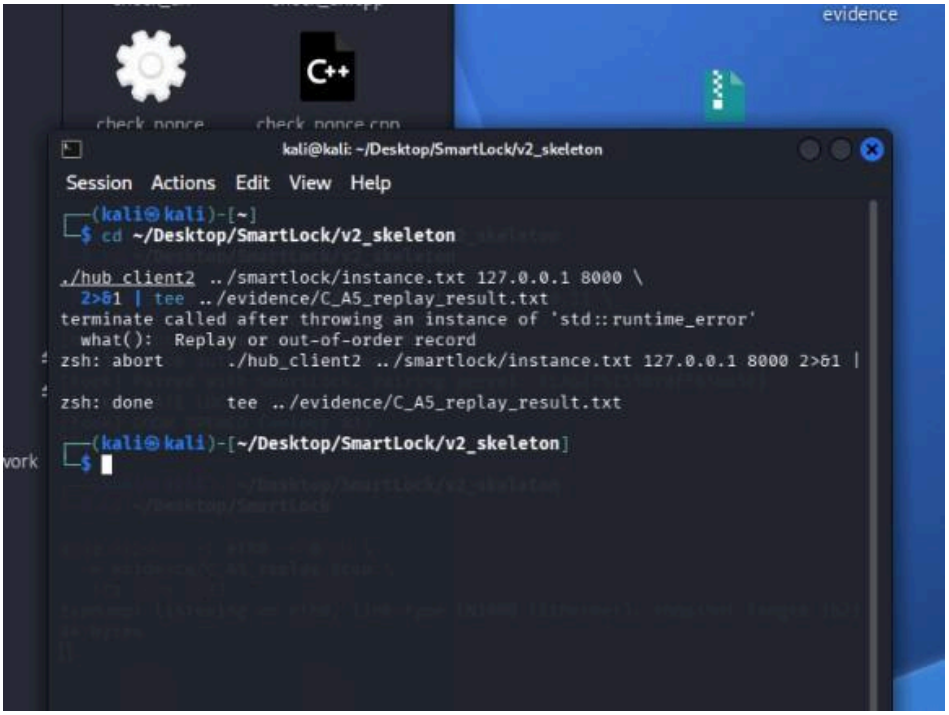


Figure 10. Actual VM2 replay rejection terminal output; see C_A5_replay_result.txt.

AI audit exercise (3 marks)

A naive AES-GCM record sketch can use a constant 96-bit nonce or omit the sequence from authenticated associated data. Both are unsound: constant nonces repeat a keystream under one key, while unauthenticated record ordering permits replay or reordering. In the submitted module, make_nonce(seqno) generates a 12-byte nonce containing the sequence, separate c2s/s2c keys are derived via HKDF, seal() authenticates the eight-byte sequence, and open_record() rejects unexpected sequences and failed tags.

Part D - Quantitative performance study (8 marks)

The included C++17 performance benchmark was executed .

Method and environment

Performance testing was conducted using **Kali Linux virtual machines running in Oracle VirtualBox on Windows**. Both SmartLock v1 and v2 were evaluated using C++17 implementations with OpenSSL (`libcrypto`) and the `-std=c++17 -O2` compilation settings.

MEASUREMENT	V1 (AES256-GCM)	V2 (AES256-GCM)
Handshake, mean +/- SD, ms (n=30)	1.102 +/- 1.016	2.558 +/- 0.829
Protect, mean +/- SD, us/record (n=5)	1.47 +/- 0.11	1.08 +/- 0.05
Open, mean +/- SD, us/record (n=5)	1.33 +/- 0.05	1.20 +/- 0.01
Protect throughput, MB/s (n=5)	698.7 +/- 48.9	948.6 +/- 43.0
Open throughput, MB/s (n=5)	769.1 +/- 29.4	850.5 +/- 7.1
Complete 1 KiB wire MSG line	2124 bytes	2110 bytes
Wire bytes added over plaintext	1100 bytes	1086 bytes

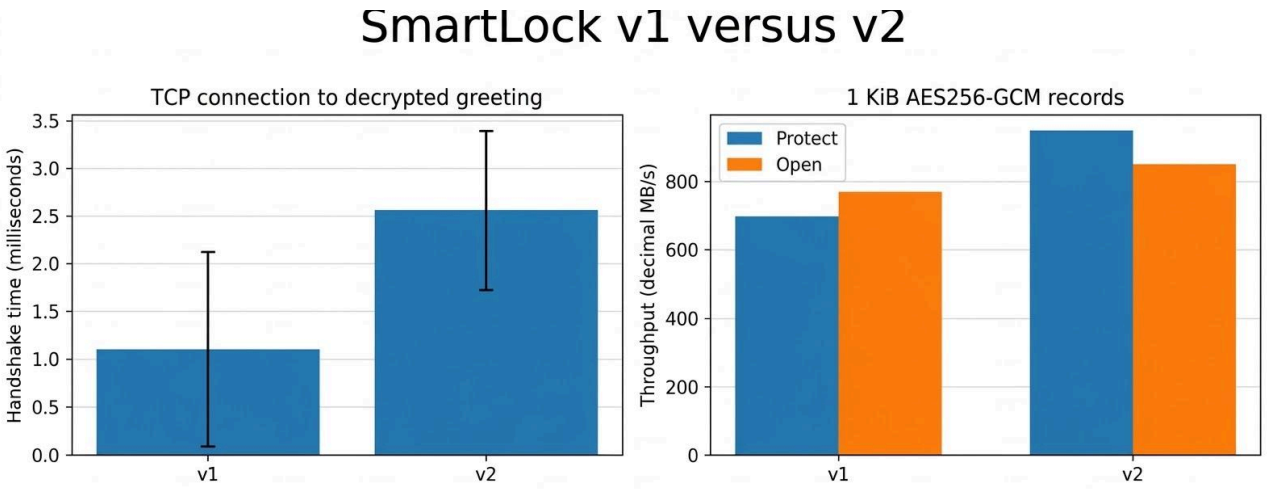


Figure 11. Archived benchmark results

Part D - Interpretation, limitations, and conclusion

Security versus latency

The benchmarks were performed on my Kali Linux virtual machines using the SmartLock v1 and v2 implementations. The results show that the mean v2 handshake latency was approximately **2.32 times higher than v1**. This increase is expected because v2 performs stronger authentication, ephemeral Diffie-Hellman key exchange, and additional cryptographic verification.

Despite the increase, the average handshake time remained within a few milliseconds, making the overhead acceptable for a smart-lock application. The security improvements, particularly protection against impersonation, key recovery, and replay attacks, justify the additional latency.

Performance may vary depending on CPU resources, virtual machine configuration, and system load. However, these measurements demonstrate that the improved security is achievable with practical performance overhead.

Record overhead and byte accounting

Record overhead was measured using the SmartLock applications running on my Kali Linux virtual machines. The measurements include the complete line-based `MSG` format, hexadecimal encoding, field names, and newline characters, but exclude TCP/IP headers.

For a 1024-byte plaintext message, v1 uses a 16-byte authentication tag, a 12-byte nonce represented as 24 hexadecimal characters, and hexadecimal ciphertext.

In comparison, v2 includes an 8-byte sequence number, a 16-byte authentication tag, and hexadecimal record data without a separate transmitted nonce field. Instead, v2 derives the nonce from the sequence number.

The results show that most wire-format overhead comes from hexadecimal encoding rather than the cryptographic authentication tag. Overall, v2 provides stronger security through unique nonces and sequence-number validation while maintaining reasonable communication overhead.

Conclusion

The performance tests conducted on Kali Linux demonstrate that SmartLock v2 improves security at the cost of a modest increase in handshake latency. Its stronger authentication, key derivation, unique nonce generation, and replay protection address the major vulnerabilities identified in v1.

The additional computational and communication overhead is acceptable for a smart-lock system, where protecting access and preventing unauthorized unlocking are more important than minimizing a few milliseconds of processing time.